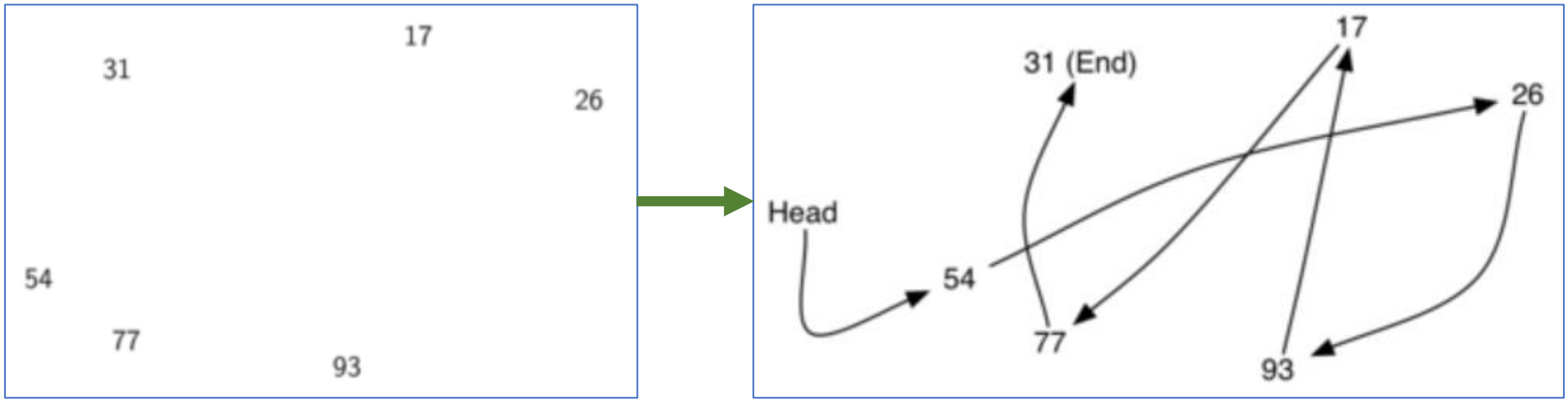
Section 11: Linked List Extension

Xiaofan Chen

x.chen3@massey.ac.nz

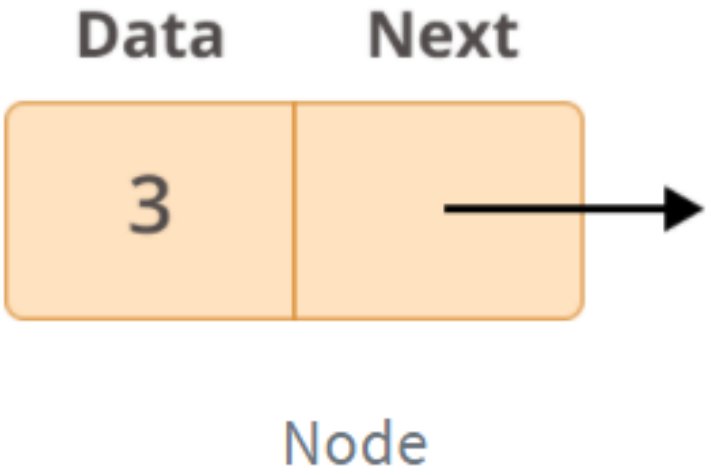
Items placed randomly → Maintained by Explicit Links



https://runestone.academy/runestone/books/published/pythonds/BasicDS/ImplementinganUnorderedListLinkedLists.html

Linked list: Node

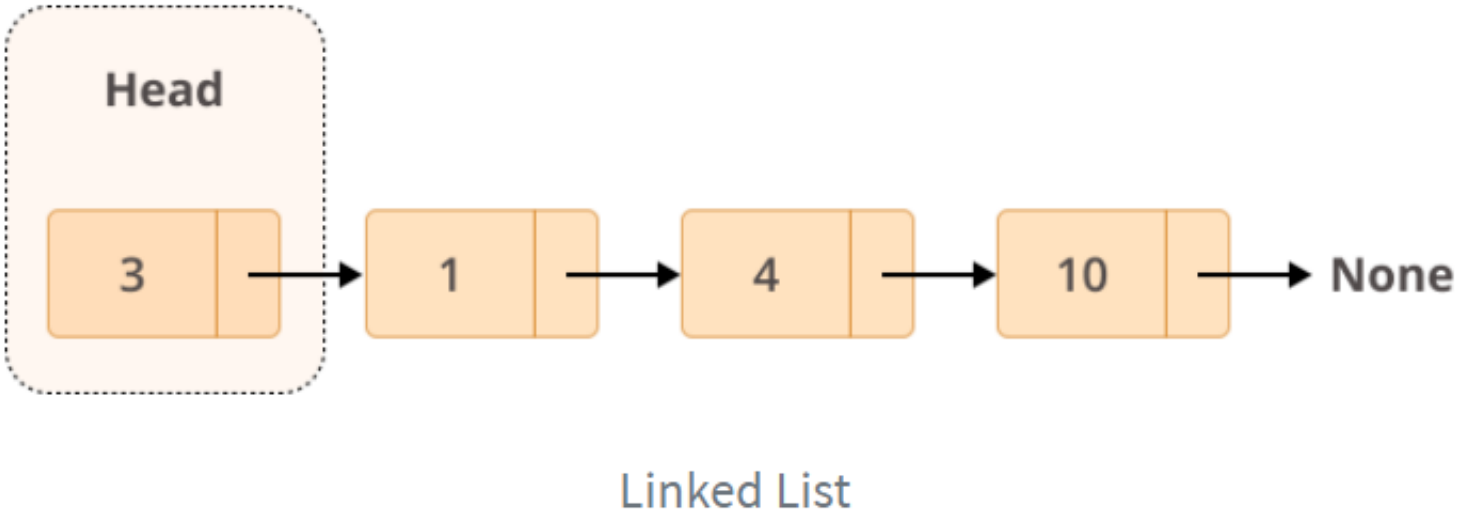
- **Data** contains the value to be stored in the node.
- **Next** contains a reference to the next node on the list.



https://realpython.com/linked-lists-python/

Linked list

A linked list is a collection of nodes. The first node is called the **head**. The last node must have its next reference pointing to **None** to determine the end of the list.

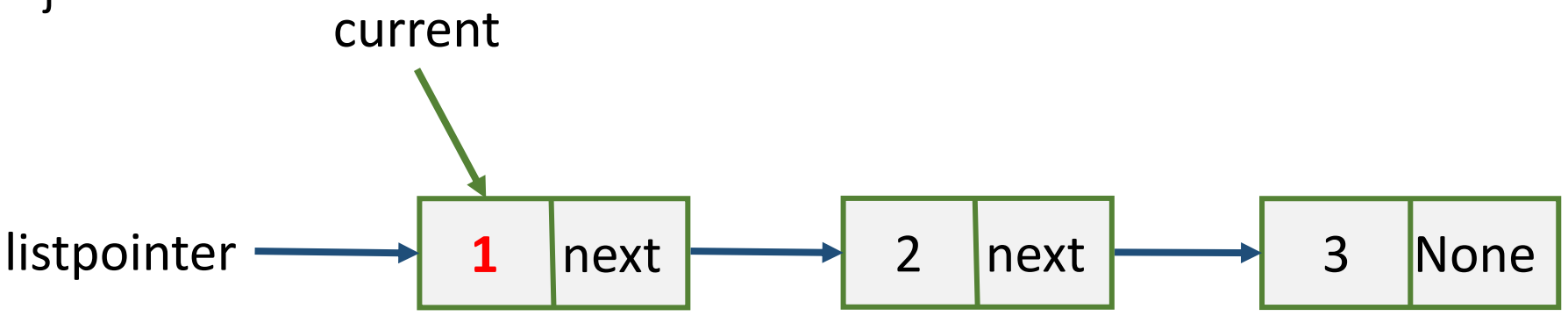


Linked List

https://realpython.com/linked-lists-python/

Traversing a linked list: using iteration

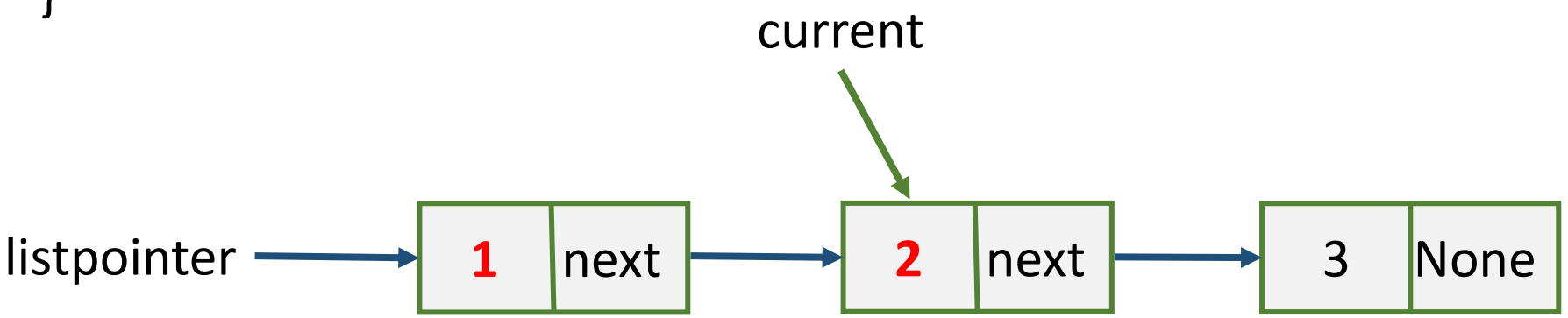
```
void displaylist() {
        Node* current;
        current = listpointer;  // pointing to the first node
        while (current != NULL) {
                cout << current->getvalue() << ", ";
                current = current->getnext();
        }
}
```



Output:

Traversing a linked list : using iteration

```
void displaylist() {
        Node* current;
        current = listpointer;  // pointing to the first node
        while (current != NULL) {
                cout << current->getvalue() << ", ";
                current = current->getnext();
        }
}
```

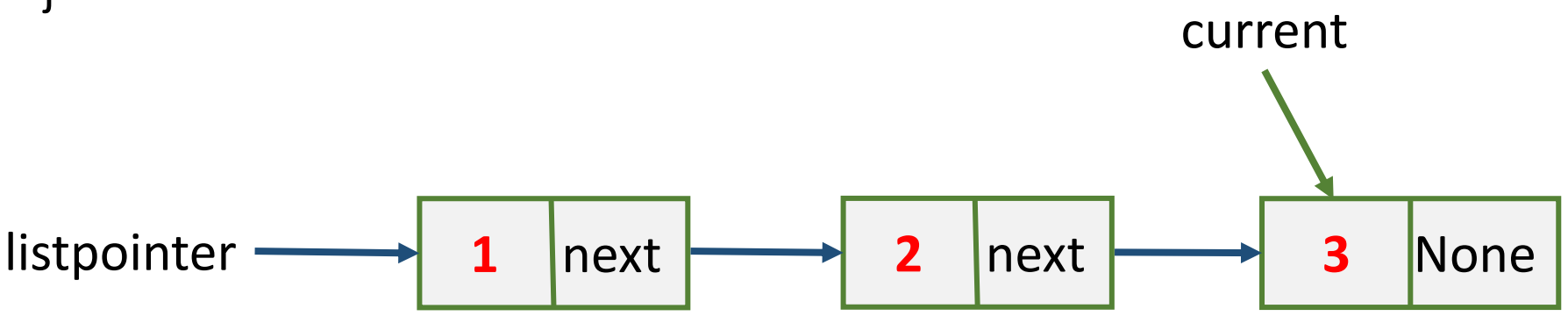


Output:

1,

Traversing a linked list : using iteration

```
void displaylist() {
        Node* current;
        current = listpointer;  // pointing to the first node
        while (current != NULL) {
                cout << current->getvalue() << ", ";
                current = current->getnext();
        }
}
```

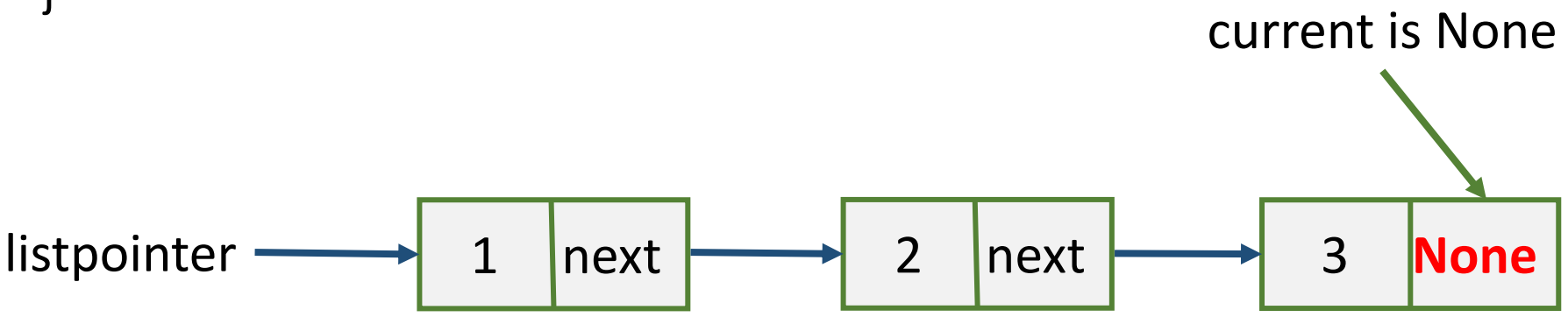


Output:

1, 2,

Traversing a linked list : using iteration

```
void displaylist() {
        Node* current;
        current = listpointer;  // pointing to the first node
        while (current != NULL) {
                cout << current->getvalue() << ", ";
                current = current->getnext();
        }
}
```



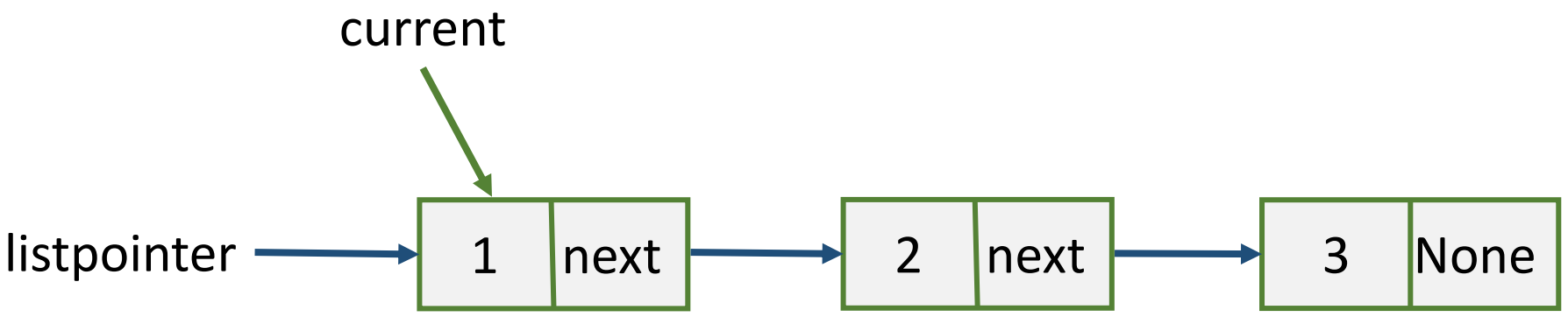
Output:

1, 2, 3,

Traversing a linked list: using recursion

```
void recursiondisplaylist(Node* current) {
    if(current == NULL){
        return;
    }
    recursiondisplaylist(current->getnext());
    cout << current->getvalue() << ", ";  // print items in reverse order
}
```

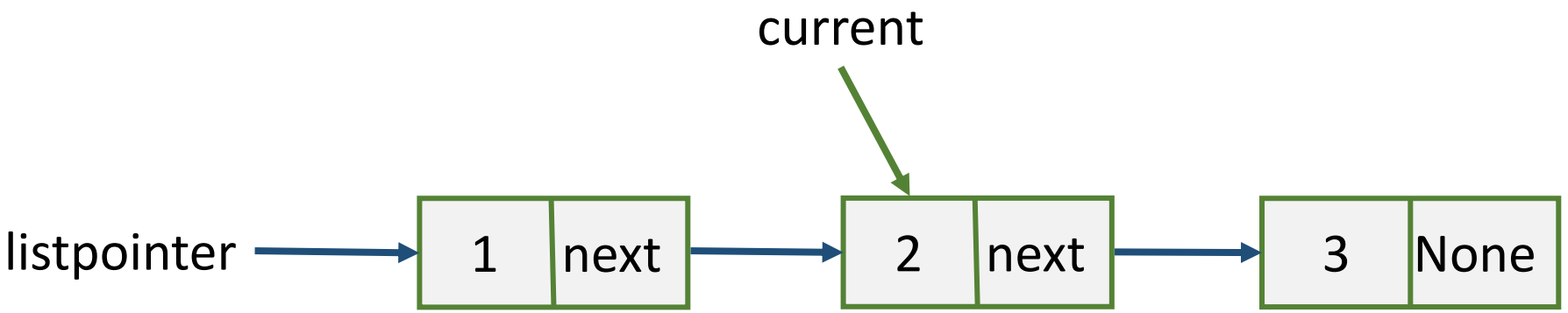
recursiondisplaylist(listpointer);



Output:

Traversing a linked list: using recursion

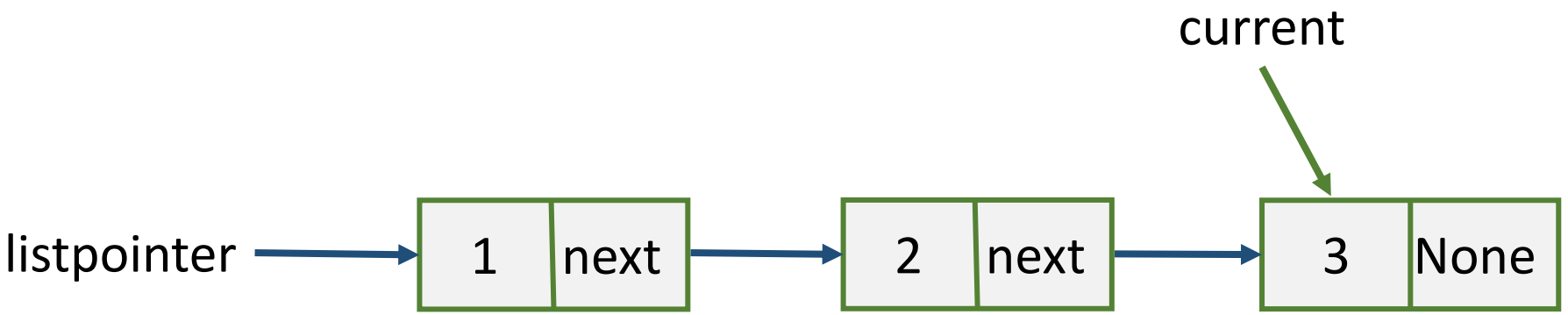
```
void recursiondisplaylist(Node* current) {
    if(current == NULL){
        return;
    }
    recursiondisplaylist(current->getnext());
    cout << current->getvalue() << ", ";  // print items in reverse order
}
recursiondisplaylist(listpointer);
```



Output:

Traversing a linked list: using recursion

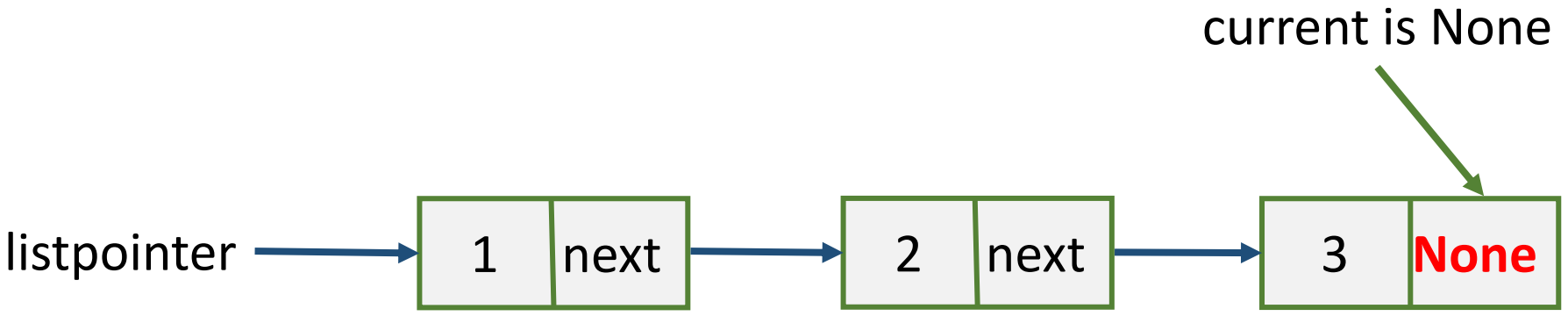
```
void recursiondisplaylist(Node* current) {
    if(current == NULL){
        return;
    }
    recursiondisplaylist(current->getnext());
    cout << current->getvalue() << ", ";  // print items in reverse order
}
recursiondisplaylist(listpointer);
```



Output:

Traversing a linked list: using recursion

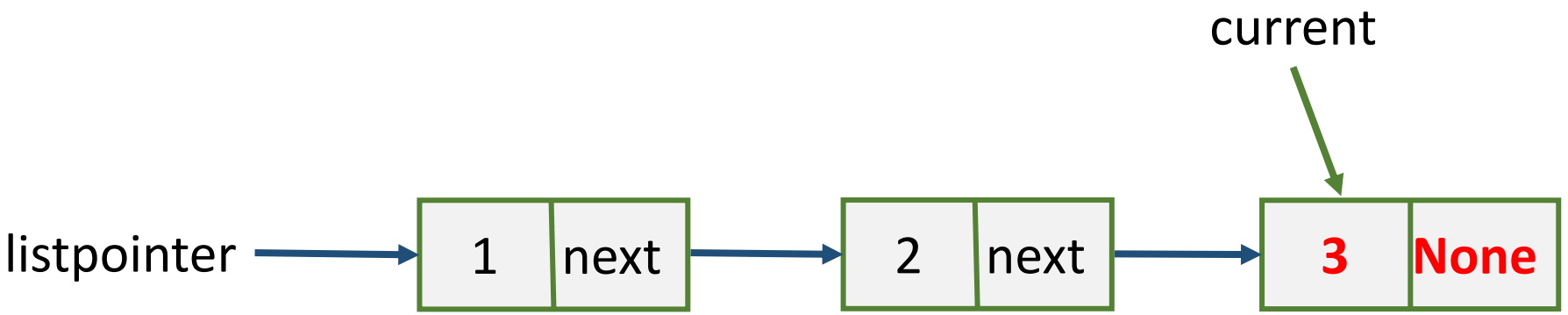
```
void recursiondisplaylist(Node* current) {
    if(current == NULL){
        return;
    }
    recursiondisplaylist(current->getnext());
    cout << current->getvalue() << ", ";  // print items in reverse order
}
recursiondisplaylist(listpointer);
```



Output:

Traversing a linked list: using recursion

```
void recursiondisplaylist(Node* current) {
    if(current == NULL){
        return;
    }
    recursiondisplaylist(current->getnext());
    cout << current->getvalue() << ", ";  // print items in reverse order
}
recursiondisplaylist(listpointer);
```

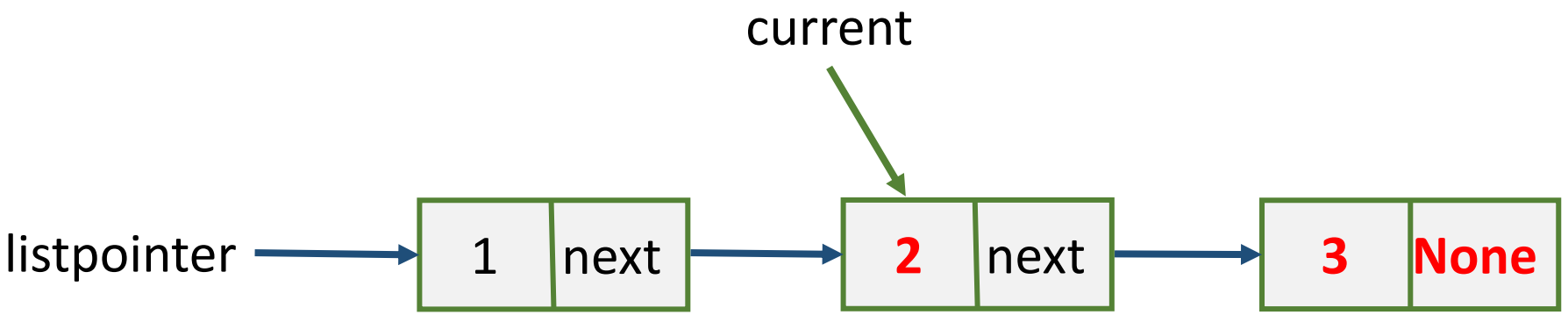


Output:

3,

Traversing a linked list: using recursion

```
void recursiondisplaylist(Node* current) {
    if(current == NULL){
        return;
    }
    recursiondisplaylist(current->getnext());
    cout << current->getvalue() << ", ";  // print items in reverse order
}
recursiondisplaylist(listpointer);
```

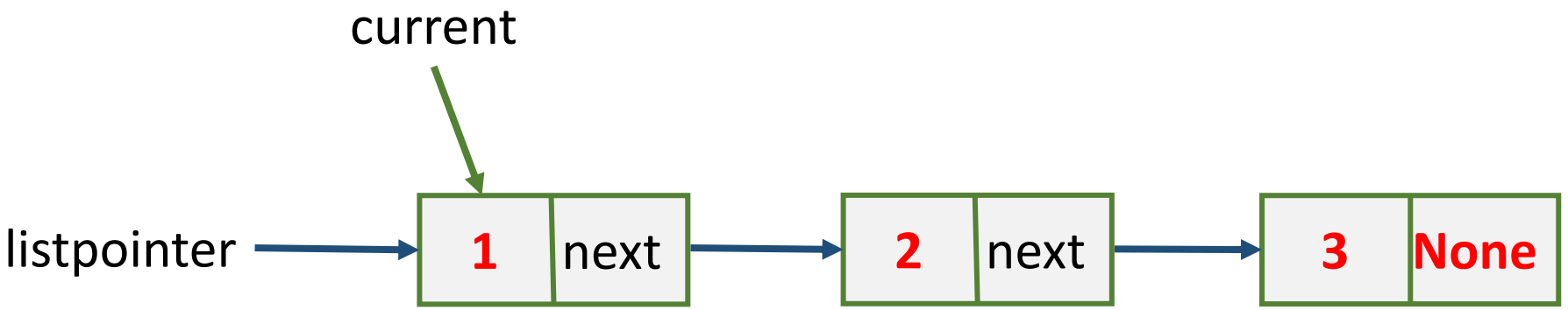


Output:

3, 2,

Traversing a linked list: using recursion

```
void recursiondisplaylist(Node* current) {
        if(current == NULL){
            return;
        }
        recursiondisplaylist(current->getnext());
        cout << current->getvalue() << ", ";  // print items in reverse order
}
recursiondisplaylist(listpointer);
```



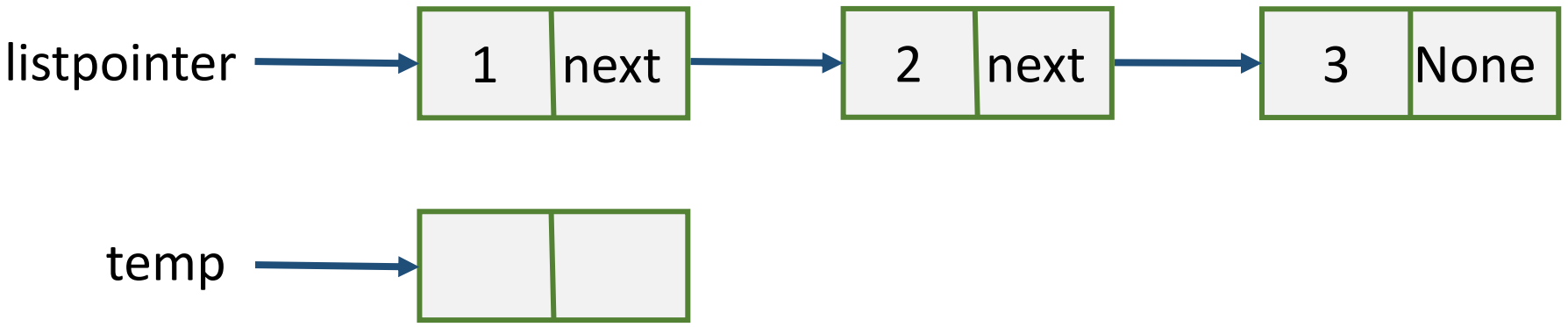
Output:

3, 2, 1,

Linked list: add to beginning

```
void addToBeginning(int value){
    Node *temp = new Node;
    temp->loaddata(value, listpointer);
    listpointer = temp;
}
```

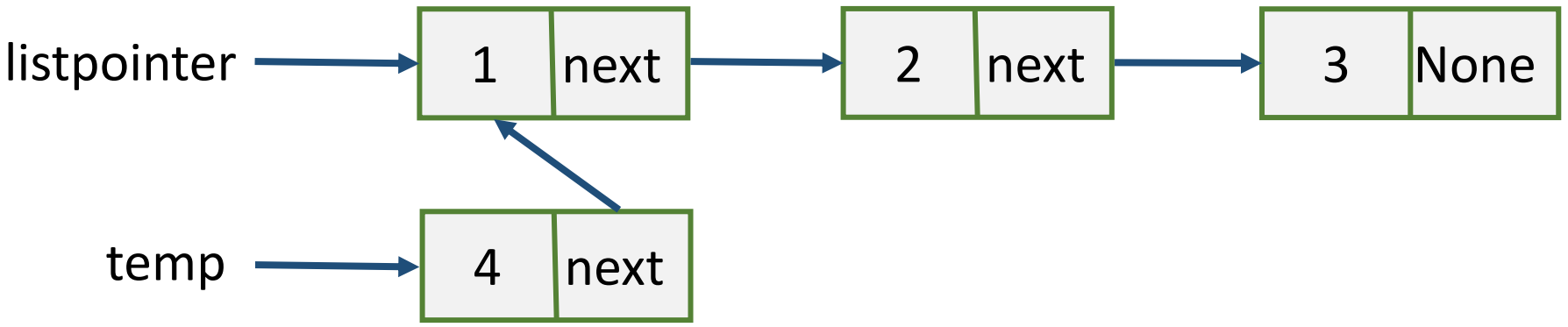
addToBeginning(4);



Linked list: add to beginning

```
void addToBeginning(int value){
    Node *temp = new Node;
    temp->loaddata(value, listpointer);
    listpointer = temp;
}
```

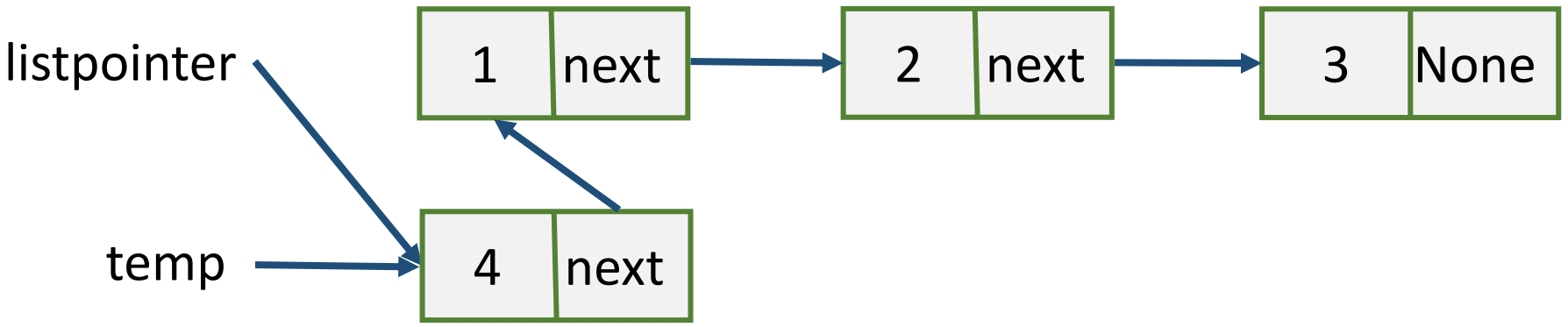
addToBeginning(4);



Linked list: add to beginning

```
void addToBeginning(int value){
    Node *temp = new Node;
    temp->loaddata(value, listpointer);
    listpointer = temp;
}
```

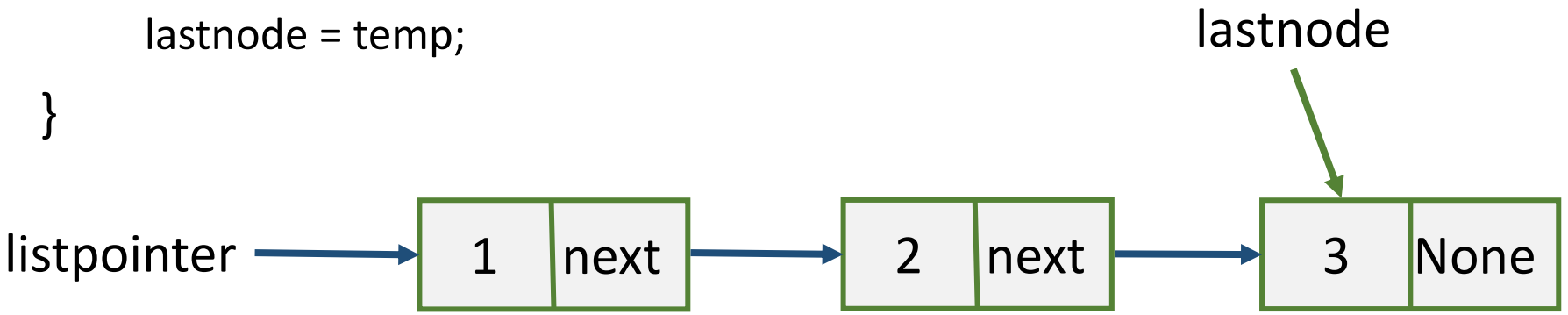
addToBeginning(4);
displaylist() output:
4, 1, 2, 3,



Linked list: add to end

```
void addToEnd(int value){
        Node* lastnode = listpointer;
        while (lastnode->getnext() != NULL) {
                lastnode = lastnode->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, NULL);
        lastnode->setnext(temp);
        lastnode = temp;
}
```

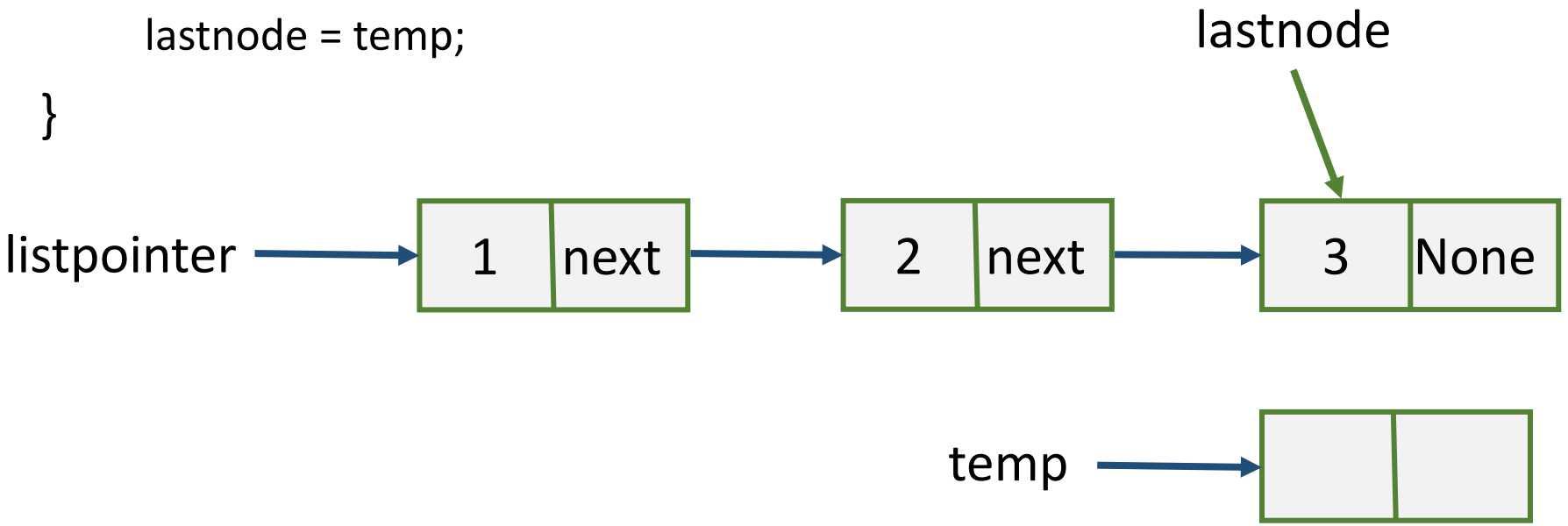
```
addToEnd(4);
```



Linked list: add to end

```
void addToEnd(int value){
        Node* lastnode = listpointer;
        while (lastnode->getnext() != NULL) {
                lastnode = lastnode->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, NULL);
        lastnode->setnext(temp);
        lastnode = temp;
}
```

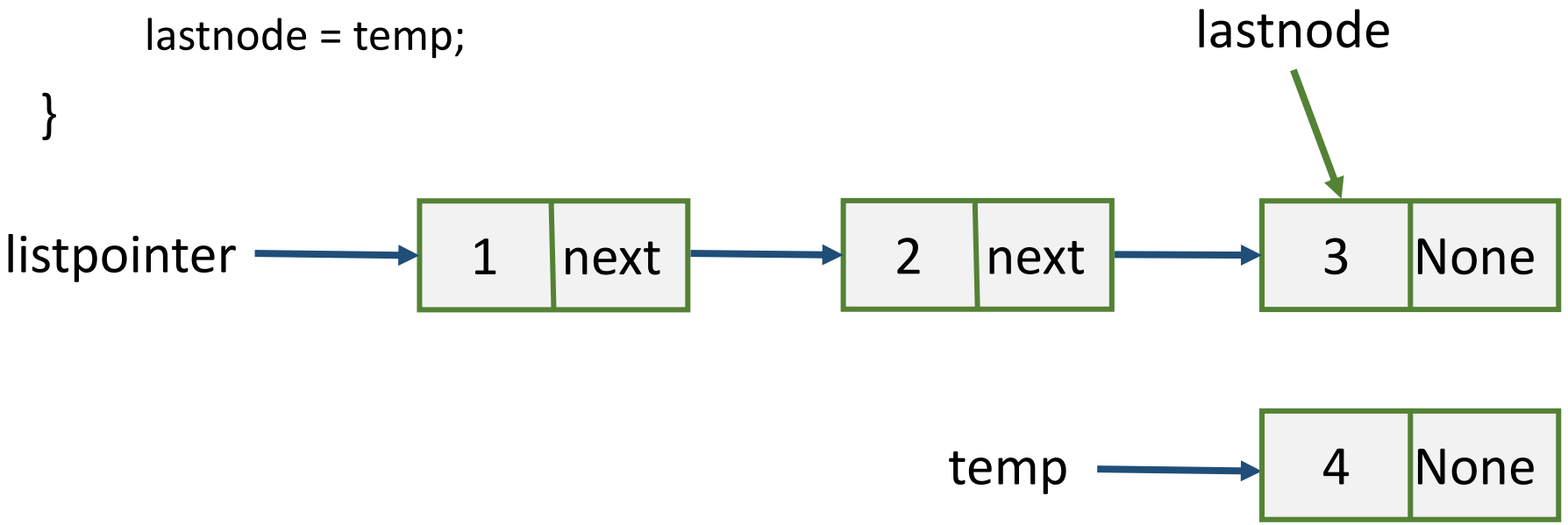
addToEnd(4);



Linked list: add to end

```
void addToEnd(int value){
        Node* lastnode = listpointer;
        while (lastnode->getnext() != NULL) {
                lastnode = lastnode->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, NULL);
        lastnode->setnext(temp);
        lastnode = temp;
}
```

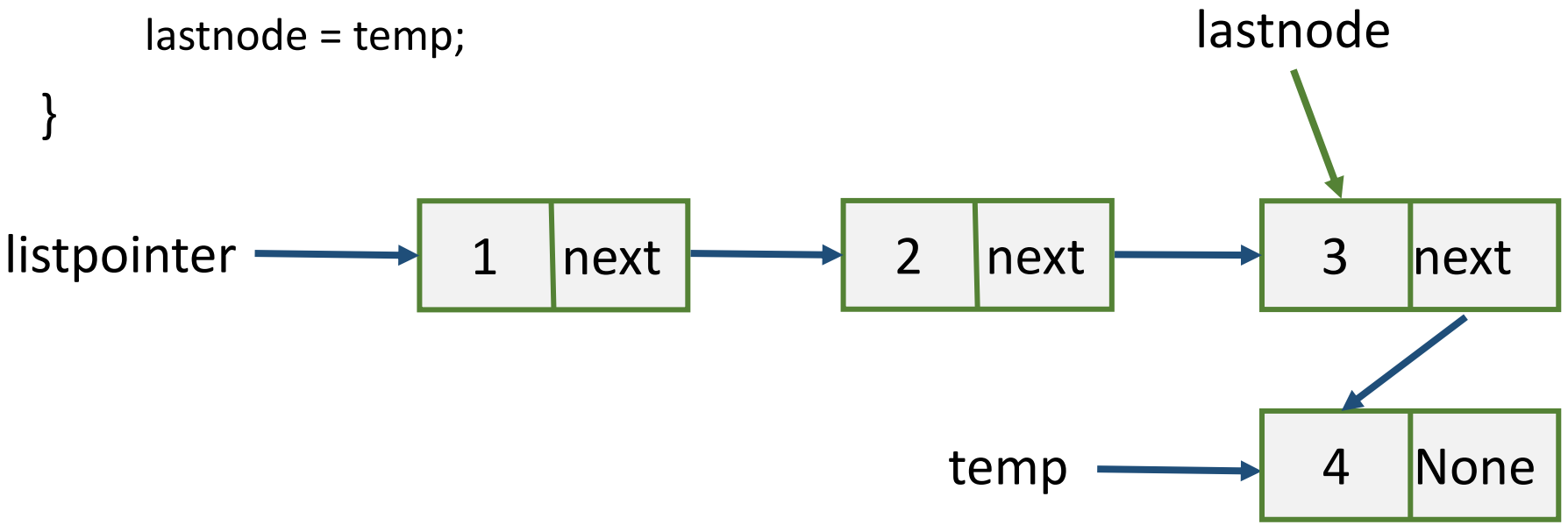
addToEnd(4);



Linked list: add to end

```
void addToEnd(int value){
        Node* lastnode = listpointer;
        while (lastnode->getnext() != NULL) {
                lastnode = lastnode->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, NULL);
        lastnode->setnext(temp);
        lastnode = temp;
}
```

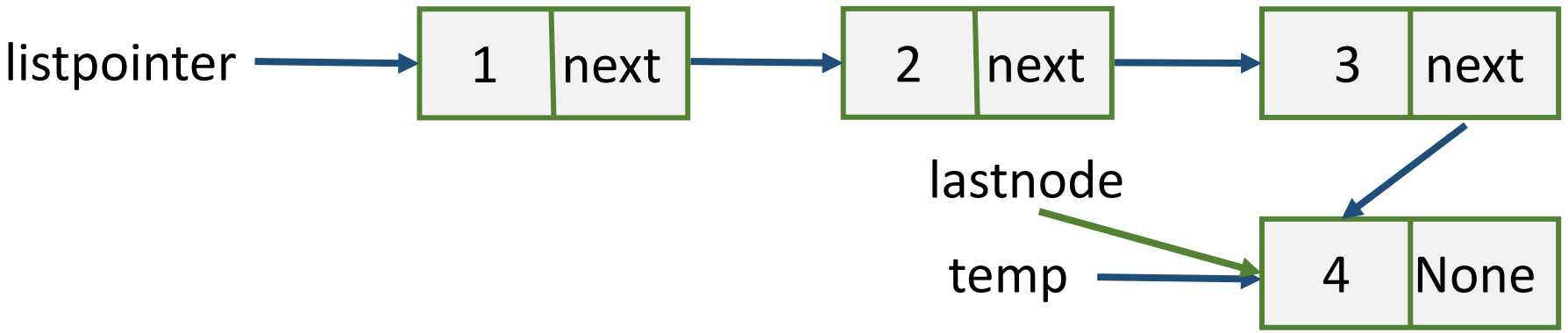
```
addToEnd(4);
```



Linked list: add to end

```
void addToEnd(int value){
        Node* lastnode = listpointer;
        while (lastnode->getnext() != NULL) {
                lastnode = lastnode->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, NULL);
        lastnode->setnext(temp);
        lastnode = temp;
}
```

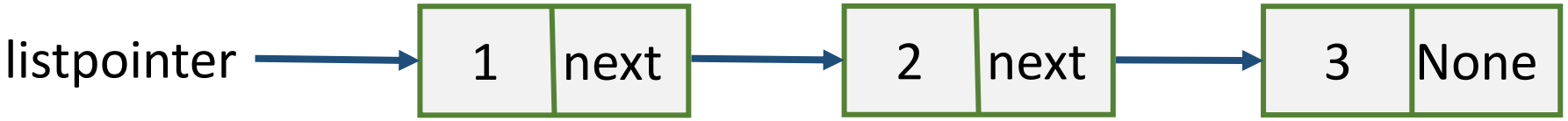
addToEnd(4);
displaylist() output:
1, 2, 3, 4,



Linked list: insert after a node

```
void insertAfter(int value, int nodeNumber){
        int i = 0;
        Node* nodebefore;
        Node* nodeafter;
        Node* current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber){
                        nodebefore = current;
                        nodeafter = current->getnext();
                        break;
                }
                current = current->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, nodeafter);
        nodebefore->setnext(temp);
}
```

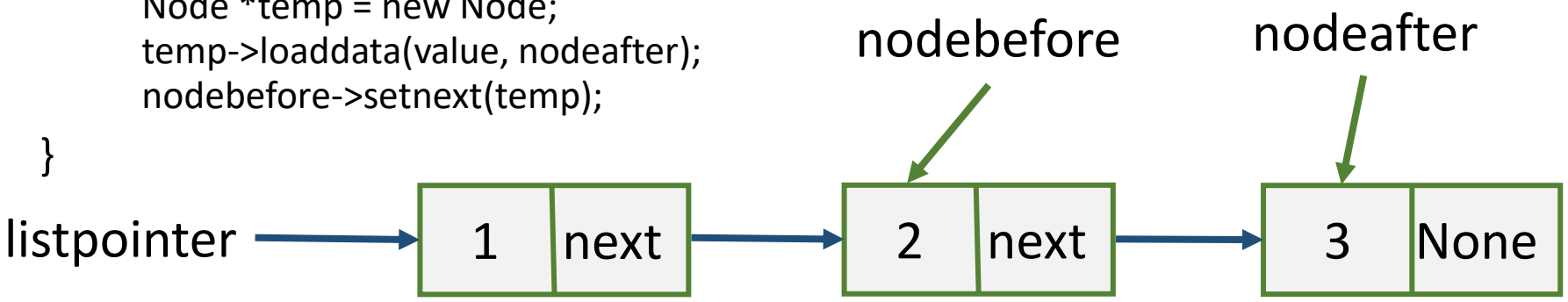
```
insertAfter(4, 2);
```



Linked list: insert after a node

```
void insertAfter(int value, int nodeNumber){
        int i = 0;
        Node* nodebefore;
        Node* nodeafter;
        Node* current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber){
                        nodebefore = current;
                        nodeafter = current->getnext();
                        break;
                }
                current = current->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, nodeafter);
        nodebefore->setnext(temp);
}
```

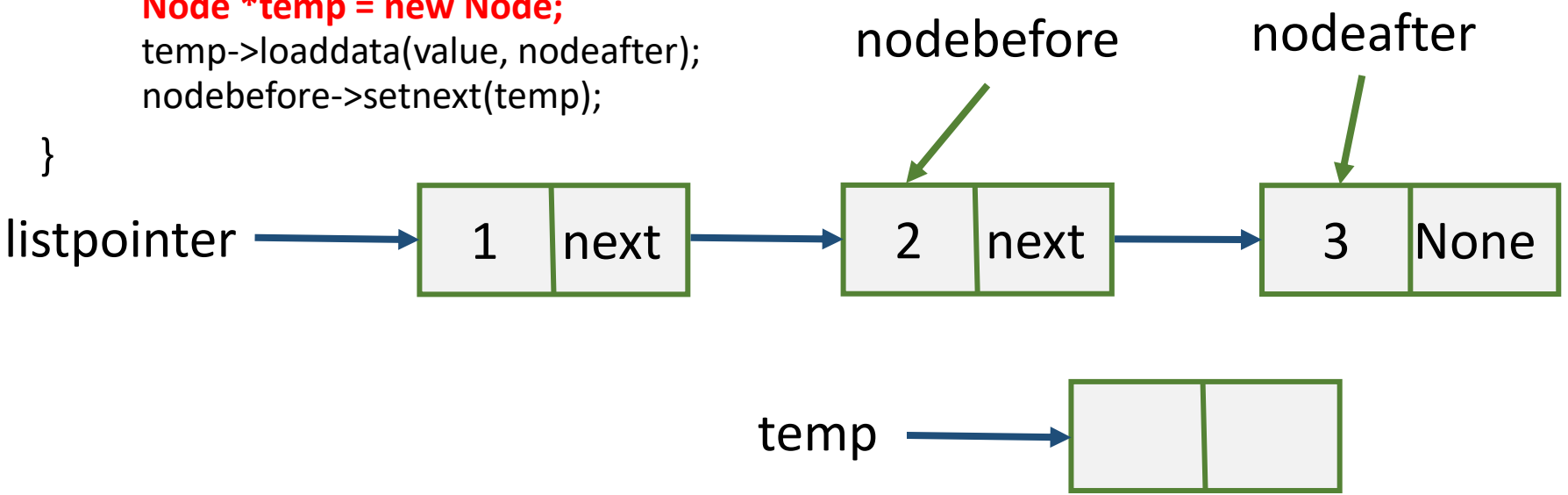
insertAfter(4, 2);



Linked list: insert after a node

```
void insertAfter(int value, int nodeNumber){
        int i = 0;
        Node* nodebefore;
        Node* nodeafter;
        Node* current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber){
                        nodebefore = current;
                        nodeafter = current->getnext();
                        break;
                }
                current = current->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, nodeafter);
        nodebefore->setnext(temp);
}
```

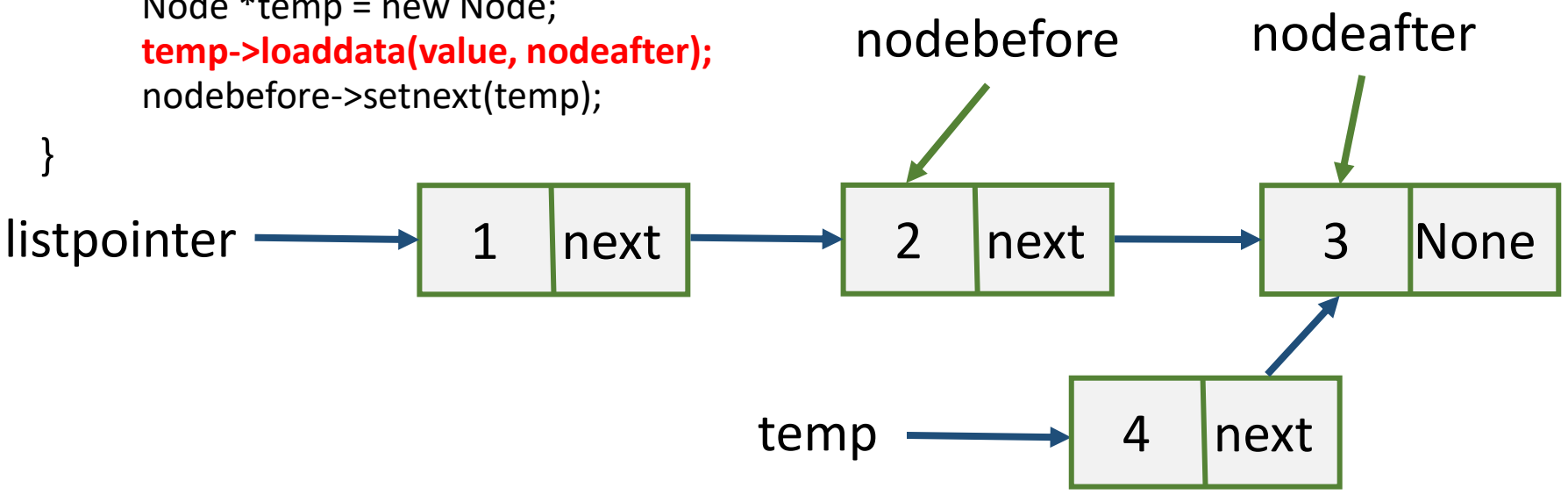
insertAfter(4, 2);



Linked list: insert after a node

```
void insertAfter(int value, int nodeNumber){
        int i = 0;
        Node* nodebefore;
        Node* nodeafter;
        Node* current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber){
                        nodebefore = current;
                        nodeafter = current->getnext();
                        break;
                }
                current = current->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, nodeafter);
        nodebefore->setnext(temp);
}
```

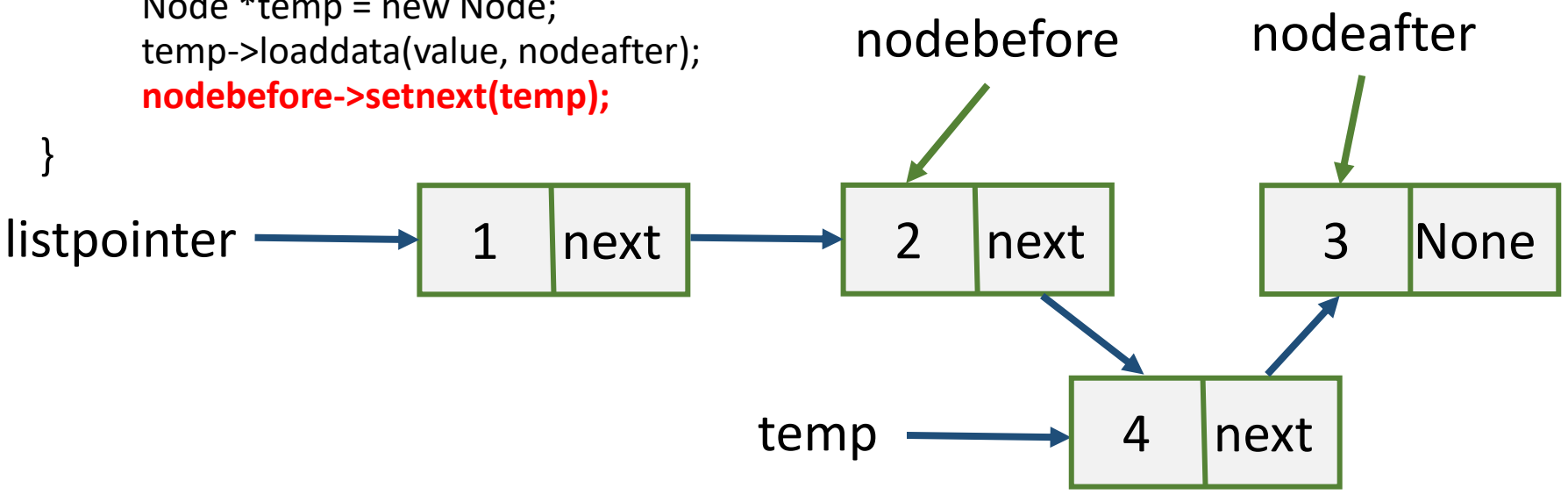
insertAfter(4, 2);



Linked list: insert after a node

```
void insertAfter(int value, int nodeNumber){
        int i = 0;
        Node* nodebefore;
        Node* nodeafter;
        Node* current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber){
                        nodebefore = current;
                        nodeafter = current->getnext();
                        break;
                }
                current = current->getnext();
        }
        Node *temp = new Node;
        temp->loaddata(value, nodeafter);
        nodebefore->setnext(temp);
}
```

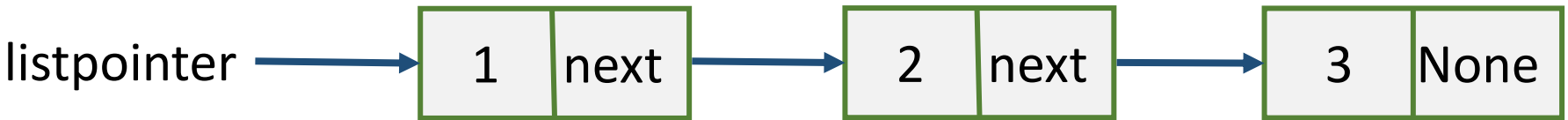
insertAfter(4, 2);
displaylist() output:
1, 2, 4, 3,



Linked list: remove a node

```
void removeNode(int nodeNumber){
        int i = 0;
        Node *nodebefore, *nodeafter, *temp, *current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber-1){
                  nodebefore = current;
                }
                if(i == nodeNumber){
                  temp = current;
                  if(nodeNumber == 1){
                    listpointer = temp->getnext();
                    delete temp;
                    return;
                  }
                }
                if(i == nodeNumber+1){
                  nodeafter = current;
                }
                current = current->getnext();
        }
        nodebefore->setnext(nodeafter);
        delete temp;
}
```

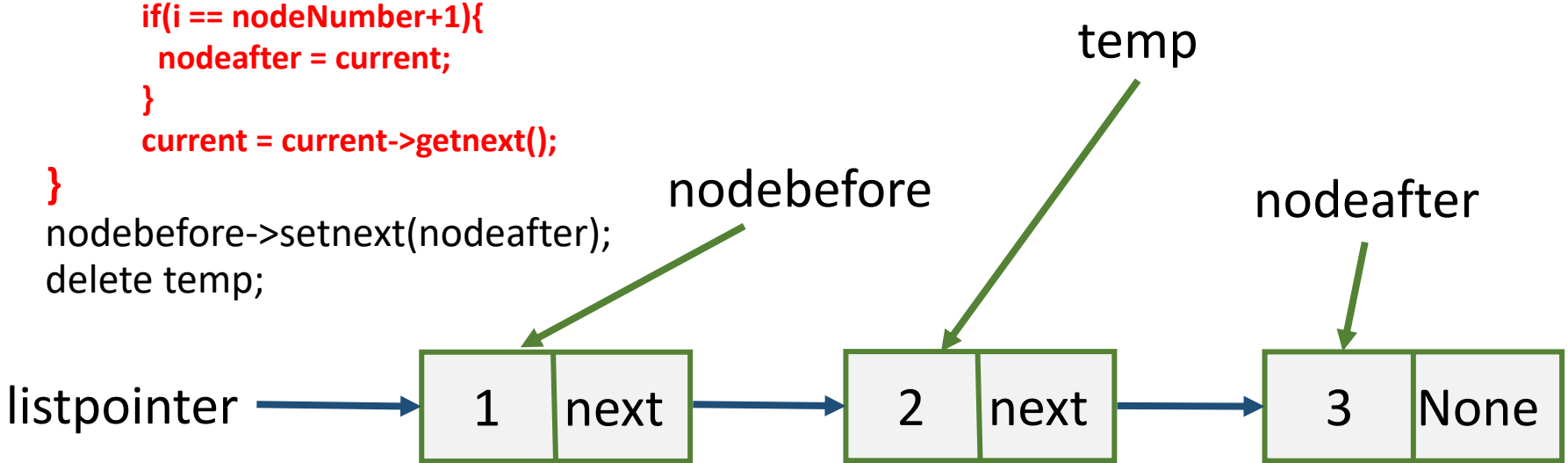
```
removeNode(2);
```



Linked list: remove a node

```
void removeNode(int nodeNumber){
        int i = 0;
        Node *nodebefore, *nodeafter, *temp, *current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber-1){
                  nodebefore = current;
                }
                if(i == nodeNumber){
                  temp = current;
                  if(nodeNumber == 1){
                    listpointer = temp->getnext();
                    delete temp;
                    return;
                  }
                }
                if(i == nodeNumber+1){
                  nodeafter = current;
                }
                current = current->getnext();
        }
        nodebefore->setnext(nodeafter);
        delete temp;
}
```

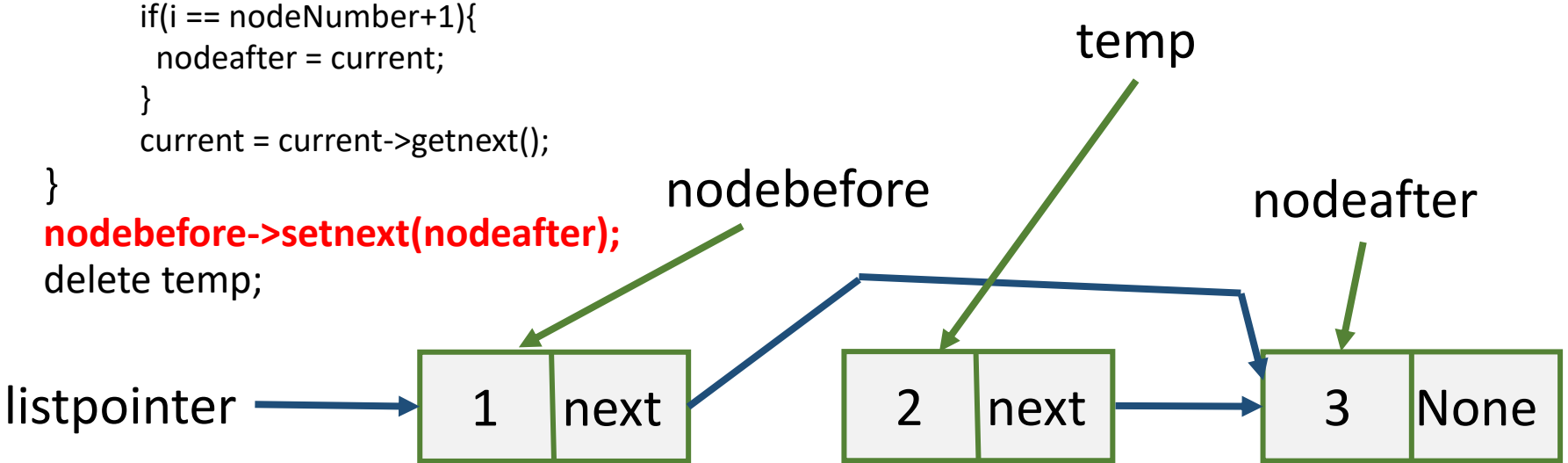
removeNode(2);



Linked list: remove a node

```
void removeNode(int nodeNumber){
        int i = 0;
        Node *nodebefore, *nodeafter, *temp, *current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber-1){
                  nodebefore = current;
                }
                if(i == nodeNumber){
                  temp = current;
                  if(nodeNumber == 1){
                    listpointer = temp->getnext();
                    delete temp;
                    return;
                  }
                }
                if(i == nodeNumber+1){
                  nodeafter = current;
                }
                current = current->getnext();
        }
        nodebefore->setnext(nodeafter);
        delete temp;
}
```

removeNode(2);



Linked list: remove a node

```
void removeNode(int nodeNumber){
        int i = 0;
        Node *nodebefore, *nodeafter, *temp, *current = listpointer;
        while (current != NULL) {
                i++;
                if(i == nodeNumber-1){
                  nodebefore = current;
                }
                if(i == nodeNumber){
                  temp = current;
                  if(nodeNumber == 1){
                    listpointer = temp->getnext();
                    delete temp;
                    return;
                  }
                }
                if(i == nodeNumber+1){
                  nodeafter = current;
                }
                current = current->getnext();
        }
        nodebefore->setnext(nodeafter);
        delete temp;
}
```

removeNode(2);
displaylist() output:
1, 3,

